

Multiparameter Persistent Homology vs. Classical Baselines

A Three-Seed Comparison on Thomas and Nested-Thomas Point Processes

point-process-tda

August 10, 2026

Contents

1	Overview	1
2	Reading the loss numbers	1
2.1	Why the scale is arbitrary without context	1
2.2	From standardized loss to a physical error factor	2
3	Nested Thomas process	3
4	Thomas process	3
5	Overall comparison	6
6	Notes on comparability	6

1 Overview

This report compares four parameter-estimation methods across two point-process families, each trained for 3 seeds (9371–9373) with the same train/val/test split per seed within a process, so every method sees identical data:

- **vihrs** — classical baseline: a 1-D CNN over the centred Ripley $L(r) - r$ function plus the point count $n(x)$, reproducing Vihrs (2022).
- **pi_multik (k=5,10,15)** — this repo’s multi-scale persistence-image CNN: a shared CoordConv encoder applied independently to DTM-filtration persistence images at three density scales ($k = 5, 10, 15$), late-fused before an MLP head.
- **mph_pi** — a CNN over a genuinely *multiparameter* persistent homology image: a single joint (density, radius) bifiltration signed measure, rendered as one (H_0, H_1) image pair. Architecturally identical to **pi_multik**’s single-scale branch (CoordConv encoder + MLP head) — the only structural difference is that scale is already folded into the second filtration axis, so there is no k -axis to fuse across.
- **mph_fusion** — late fusion of **vihrs**’s $L(r) - r$ branch and **mph_pi**’s CoordConv branch, motivated by the two methods specializing in different targets (Section 3).

All targets are trained in *standardized log space*: each target is log-transformed, then z -scored using the mean/std fit on that seed’s training split; the reported loss is always mean squared error (MSE) on that standardized scale. Section 2 explains exactly what a given loss value means in the original, physical units of each parameter.

2 Reading the loss numbers

Every loss in this report is a standardized MSE, not a raw squared error in physical units — so “0.03” does not mean the same thing for every target, and does not mean anything at all without translation. This section makes that translation concrete.

2.1 Why the scale is arbitrary without context

Each target y (e.g. `parent_intensity`) is transformed before training as $z = (\log y - \mu_{\log})/\sigma_{\log}$, where $\mu_{\log}$ and $\sigma_{\log}$ are the mean and standard deviation of $\log y$ on that seed’s *training* split. The model is scored by $\text{MSE} = \mathbb{E}[(\hat{z} - z)^2]$ on the test split. Two properties fall out of this construction:

- **The “no-skill” baseline is ≈ 1 .** A model that always predicts the training mean ($\hat{z} = 0$) scores $\text{MSE} \approx \text{Var}(z) \approx 1$ (by construction, on data drawn like the training set). So loss values are naturally read against 1.0, not against 0: a loss of 0.5 means the model is explaining about half the variance in log-space, a loss of 0.03 means it is explaining almost all of it.
- $R^2 \approx 1 - \text{loss}$. This is the standard variance-explained interpretation, directly usable without further transformation.
- **The same loss value means a different real-world error for different targets**, because $\sigma_{\log}$ (how spread out that target is in log-space to begin with) differs by target. A target that already spans a narrow log-range needs a much smaller absolute log-error to reach a given standardized loss than a target that spans a wide one.

2.2 From standardized loss to a physical error factor

Undoing both transforms (z-score, then log) turns a standardized RMSE into a *multiplicative* error factor in the original units: if $\text{RMSE}_z = \sqrt{\text{loss}}$, then a “typical” prediction sits within a factor of $\exp(\text{RMSE}_z \cdot \sigma_{\log})$ of the true value. Tables 1 and 2 apply this to every target actually used in this report, at a range of representative loss values.

Loss	R^2	parent_intensity	meta_offspring	meta_cluster_scale	mean_offspring	cluster_scale
0.50	0.50	42%	34%	56%	44%	59%
0.20	0.80	25%	20%	32%	26%	34%
0.10	0.90	17%	14%	22%	18%	23%
0.05	0.95	12%	10%	15%	12%	16%
0.03	0.97	9%	7%	11%	9%	12%
0.01	0.99	5%	4%	6%	5%	7%

Table 1: Nested Thomas: standardized loss $\rightarrow R^2 \rightarrow$ typical multiplicative error per target (e.g. “9%” means a typical prediction is within $\pm 9\%$ of the true value).

Worked example. A loss of 0.03 on `cluster_scale` (nested Thomas, $\sigma_{\log} = 0.655$) corresponds to $R^2 = 0.97$ and a typical prediction within about $\pm 12\%$ of the true value; the same loss of 0.03 on `meta_offspring` ($\sigma_{\log} = 0.409$) — a target with less inherent log-spread — corresponds to the same R^2 but a tighter $\pm 7\%$ typical error. The take-away: compare loss numbers *within* a target/process across methods freely, but don’t compare raw loss values *across* targets without consulting Tables 1/2 first.

Loss	R^2	parent_intensity	mean_offspring	cluster_scale
0.50	0.50	50%	63%	63%
0.20	0.80	29%	36%	36%
0.10	0.90	20%	24%	24%
0.05	0.95	14%	17%	17%
0.03	0.97	11%	13%	13%
0.01	0.99	6%	7%	7%

Table 2: Thomas: standardized loss $\rightarrow R^2 \rightarrow$ typical multiplicative error per target (e.g. "9%" means a typical prediction is within $\pm 9\%$ of the true value).

3 Nested Thomas process

Nested Thomas is a two-level cluster process (meta-clusters of sub-clusters), with five targets: `parent_intensity`, `meta_offspring`, `meta_cluster_scale`, `mean_offspring`, `cluster_scale`. 8000 clouds (7000 train/val/test, 1000 held-out adversarial), $\sigma_{\log}$ per target in Table 1.

Method	Test loss [†]	Adversarial loss [†]
<code>vihrs</code>	0.2171 ± 0.0037	0.2194 ± 0.0055
<code>pi_multik</code>	0.1927 ± 0.0029	0.2000 ± 0.0017
<code>mph_pi</code>	0.2095 ± 0.0045	0.2186 ± 0.0031
<code>mph_fusion</code>	0.1965 ± 0.0022	0.1966 ± 0.0009

Table 3: Nested Thomas: overall test/adversarial loss, mean $\pm$ std over 3 seeds. [†]standardized log-MSE (Section 2), not raw physical units – see Table 1 to translate.

Method	parent_intensity	meta_offspring	meta_cluster_scale	mean_offspring	cluster_scale
<code>vihrs</code>	0.320	0.494	0.132	0.112	0.028
<code>pi_multik</code>	0.286	0.410	0.142	0.086	0.040
<code>mph_pi</code>	0.289	0.449	0.158	0.109	0.042
<code>mph_fusion</code>	0.280	0.445	0.131	0.097	0.029

Table 4: Nested Thomas: per-target test loss (standardized log-MSE, Section 2), mean over 3 seeds – see Table 1 to translate, corresponding figure for error bars.

Finding: no single method dominates every target, and the strongest overall performer is a close contest between two very different architectures. Overall (all-target) test loss ranks `pi_multik` (0.1927) $\approx$ `mph_fusion` (0.1965) $<$ `mph_pi` (0.2095) $<$ `vihrs` (0.2171) — but that ordering does not hold target-by-target. `mph_pi` (single joint bifiltration image) is clearly better than `vihrs` at the intensity/count targets (`parent_intensity`, `meta_offspring`); `vihrs`’s $L(r) - r$ is clearly better at `cluster_scale` — consistent with what each feature actually encodes: $L(r) - r$ is built to detect a characteristic clustering *radius* directly, while the DTM-bifiltration image is fundamentally a density/topology signal. `mph_fusion` (late-fusing those two) wins `parent_intensity` and `meta_cluster_scale` outright and beats both of its parent methods on overall test loss in all 3/3 seeds. `pi_multik` (three single-parameter DTM scales, $k = 5, 10, 15$, fused) takes a different route to a similarly strong result, winning `meta_offspring` and `mean_offspring` outright — sweeping several fixed density scales evidently captures much of what the joint bifiltration captures, at

nested thomas: training curves (seed 9371)

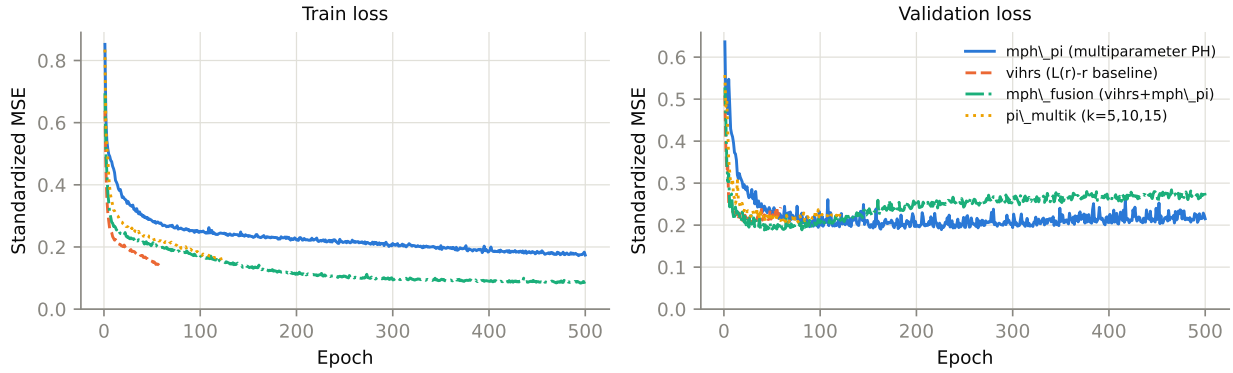


Figure 1: Training/validation loss curves, seed 9371 (representative). `vihrs` and `pi_multik` use early stopping (patience 30/50 epochs on validation loss); `mph_pi`/`mph_fusion` always run the full 500-epoch budget but reload the best-validation-loss checkpoint before final evaluation, so all four are compared on their best-checkpoint numbers regardless of curve length.

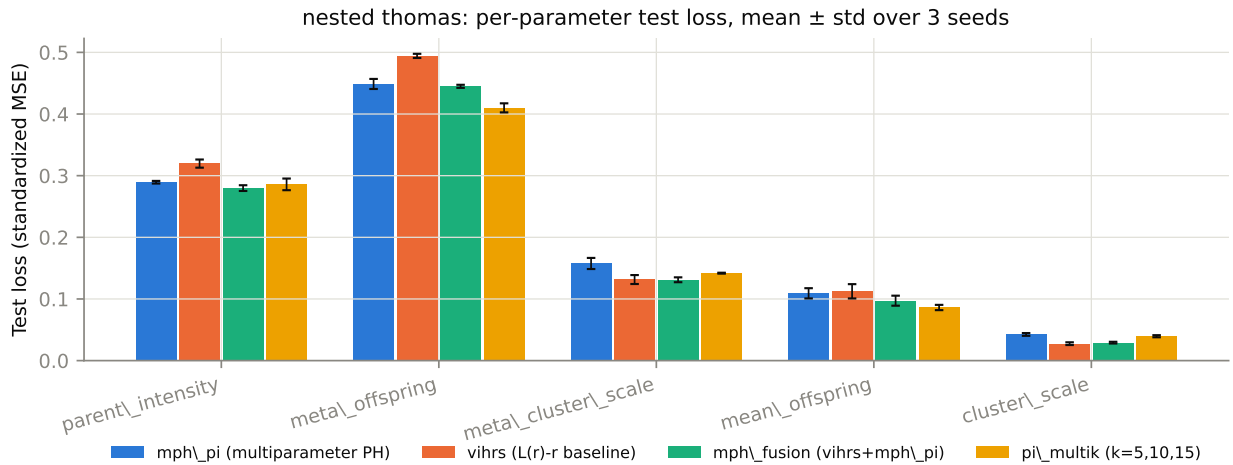


Figure 2: Per-target test loss, mean $\pm$ std over 3 seeds.

least on this process. `vihrs` alone wins only `cluster_scale`, but wins it clearly (0.028 vs. 0.029–0.043 for everything else), so it is not dominated either. The practical takeaway: the choice of TDA representation trades off *which* parameters it resolves best, not just *how well* on average, and fusion (of either kind) is the more robust choice when no single target dominates the estimation task.

4 Thomas process

Plain Thomas is a single-level cluster process, with three targets: `parent_intensity`, `mean_offspring`, `cluster_scale`. Same 8000-cloud (7000+1000) split convention as nested Thomas. `pi_multik` for this process was trained on HPC rather than locally (Section 6); everything else matches the nested-Thomas protocol exactly, same 3 seeds.

Method	Test loss [†]	Adversarial loss [†]
<code>vihrs</code>	0.1154 ± 0.0091	0.1180 ± 0.0008
<code>pi_multik</code>	0.1248 ± 0.0064	0.1233 ± 0.0027
<code>mph_pi</code>	0.1307 ± 0.0051	0.1289 ± 0.0037
<code>mph_fusion</code>	0.1131 ± 0.0097	0.1148 ± 0.0024

Table 5: Thomas: overall test/adversarial loss, mean $\pm$ std over 3 seeds. [†]standardized log-MSE (Section 2), not raw physical units – see Table 2 to translate.

Method	<code>parent_intensity</code>	<code>mean_offspring</code>	<code>cluster_scale</code>
<code>vihrs</code>	0.193	0.125	0.029
<code>pi_multik</code>	0.202	0.131	0.041
<code>mph_pi</code>	0.210	0.143	0.040
<code>mph_fusion</code>	0.188	0.123	0.029

Table 6: Thomas: per-target test loss (standardized log-MSE, Section 2), mean over 3 seeds – see Table 2 to translate, corresponding figure for error bars.

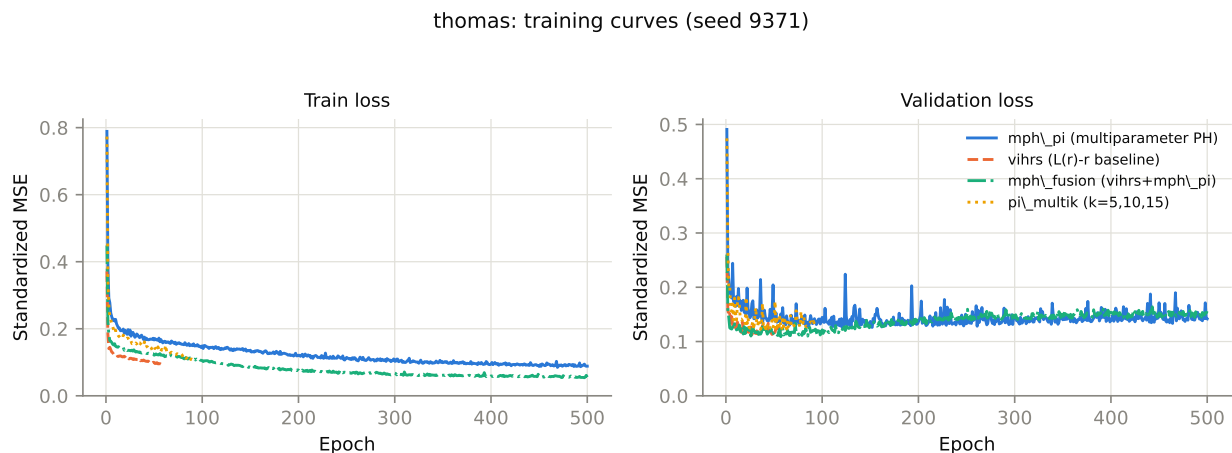


Figure 3: Training/validation loss curves, seed 9371 (representative).

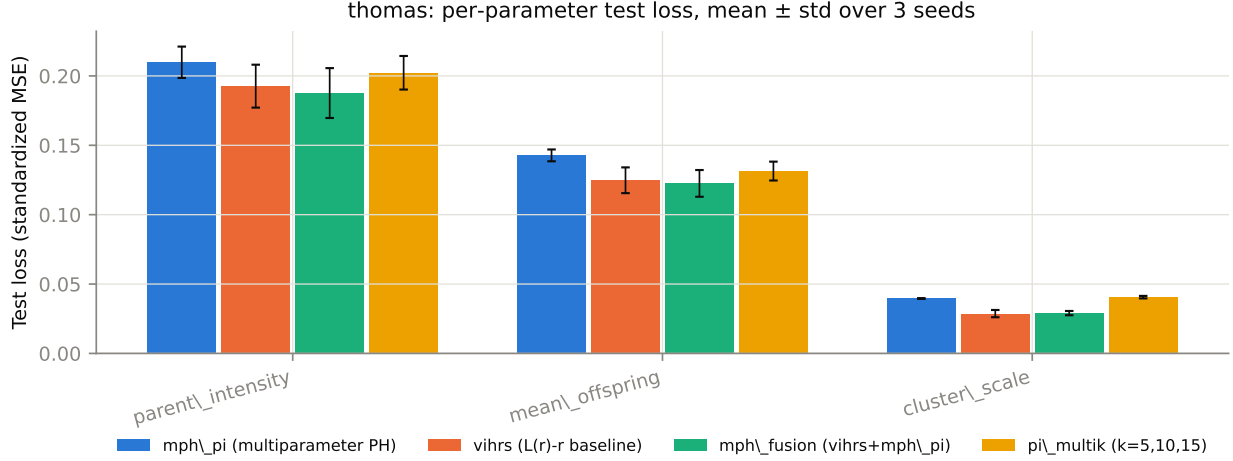


Figure 4: Per-target test loss, mean $\pm$ std over 3 seeds.

Finding: a cleaner win for fusion here than on nested Thomas. Overall test loss ranks `mph_fusion` (0.1131) < `vihrs` (0.1154) < `pi_multik` (0.1248) < `mph_pi` (0.1307). Unlike nested Thomas, `pi_multik` does not lead here — with only one clustering scale to resolve (rather than two nested ones), sweeping three DTM k values adds less on top of a single joint bifiltration or a direct radius estimate. `mph_fusion` wins `parent_intensity` and `mean_offspring` outright; `vihrs` alone still wins `cluster_scale` clearly (0.029 vs. 0.040 for the DTM-based methods) — the same pattern as nested Thomas: $L(r) - r$ remains the sharpest tool specifically for the spatial-scale target, and fusing it in rather than discarding it is what wins overall.

5 Overall comparison

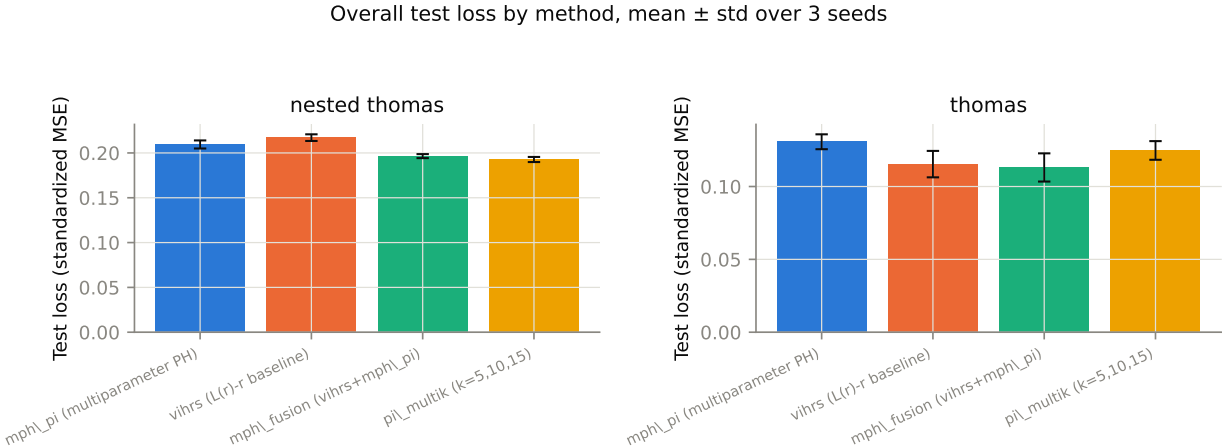


Figure 5: Overall (all-target) test loss by method, mean $\pm$ std over 3 seeds.

6 Notes on comparability

- **Training recipes differ.** `vihrs` and `pi_multik` use their own bespoke training loops (Adam/AdamW with early stopping on validation loss); `mph_pi/mph_fusion` use this repo's shared single-source training runner (plain Adam, fixed weight decay, no early stopping — always trains the full epoch budget). All four reload the best-validation-loss checkpoint before final test/adversarial evaluation, so final numbers are comparable despite the different stopping rules; the difference only costs `mph_pi/mph_fusion` extra wall-clock time; it does not advantage either side numerically.
- **Data provenance.** All numbers here come from data generated (or regenerated) after this report's underlying pipeline changes; two data issues were found and fixed while producing these results: `nested_thomas`'s $k = 5$ diagrams had never finished computing, and `thomas`'s point clouds had been lost (the $k = 5/10/15$ diagrams on disk no longer matched any reproducible cloud set) and were regenerated from the process's seeded design.
- **Compute location.** `thomas`'s `pi_multik` (3 seeds) was trained on HPC and its results downloaded rather than trained locally, after the local diagram-computation step for this method proved too memory-heavy for reliable local execution (see the repo history around `scripts/featurize.py`'s per-cloud persistent-homology computation). Every other number in this report — including `thomas`'s `vihrs/mph_pi/mph_fusion` runs, which were *also* re-obtained from that same HPC download after a local crash lost the original local copies — was validated (clean unpickle, correct label set, expected record counts) before being incorporated here, but was not independently re-run in both places to confirm byte-identical reproduction.
- **Adversarial loss** is evaluated on a held-out population generated with a disjoint seed offset, never seen during training or model selection.